

Telecom

Telecom Domain Project

API Testing for Contact List Application

Name : **Sumana Chowdhury**

Batch : **SA2510014**

Role : **Automation Trainee**

Technology : **Postman and Selenium**

Framework : **RestAssured, TestNG, Maven**

Automation Technique : **Extent Reports, Newman Reports**

Table of Contents :

1. Project Name
2. Domain
3. Application Under Testing
4. Project Description
5. Project Objectives
6. Tools and Technologies
7. Testing Flow
8. Postman Collection – Chai Assertion
9. RestAssured Automation Framework
10. Reporting
11. Conclusion

1. Project Name

API Testing for Contact List Application

2. Domain

Telecom

3. Application Under Testing

<https://thinking-tester-contact-list.herokuapp.com>

4. Project Description

This project performs end-to-end API testing for a Contact List Application. Postman was used for designing and executing API requests and validating responses using **Chai assertions**. Automated API test cases were implemented using the **RestAssured framework** in Java to verify request-response structure, status codes, headers, and response body. The testing covered key operations such as **user authentication, contact creation, update, retrieval, and deletion (CURD)**.

5. Project Objectives

- Validate API response **status codes and messages**
- Verify **Authentication and Authorization** using Bearer Token
- Ensure **CURD operations** work correctly.
- Automate APIs using **RestAssured**
- Generate **Newman** and **Extent Test Reports**
- Document test cases and results

6. Tools and Technologies

<u>Tools/Technology</u>	<u>Purpose</u>
Postman	Manual & Scripted API testing
Chai Assertions	Response Validation in Postman
Java	Programming language
RestAssured	API Automation
TestNG	Test Execution
Maven	Dependency Management
Newman and Extent Reports	HTML Test Reports

7. Testing Flow

- Add user
- Get User Profile
- Update User
- Login User
- Add Contact
- Get Contact List
- Get Contact
- Update Full Contact
- Update Partial Contact
- Logout User

8. Postman Collection – Chai Assertion

Test Case 1 : Add New User

The screenshot displays the Postman interface for a POST request to the endpoint `{{BaseUrl}}/users`. The **Scripts** tab is active, showing the following Chai assertions:

```
1 // Test for Status code
2 pm.test("Test for status code 201", ()=>{
3   pm.expect(pm.response.code).to.eql(201);
4 })
5
6 // Test for Status message : Created
7 pm.test("Test for status message : Created", ()=>{
8   pm.expect(pm.response.status).to.eql("Created");
9 })
10
11 // save generated token in environment variable
12 let tokenValue= pm.response.json().token;
13 pm.environment.set("Token1",tokenValue);
14
```

The **Test Results** tab shows the following results:

- PASSED** Test for status code 201
- PASSED** Test for status message : Created

The status bar at the bottom indicates a **201 Created** response, with a duration of 1.08 s and a size of 1.25 KB. The **Save Response** button is visible.

POST

{{BaseUrl}}/users

Docs

Params

Authorization

Headers (9)

Body

Scripts

Settings

Pre-request

Post-response

```

1 // Create dynamic email address
2 let time = new Date().getTime();
3 let email = "test123"+time+"@gmail.com"
4 console.log(email);
5 pm.environment.set("userEmail",email);
6
7

```

201 Created

1.08 s

1.25

Body

Cookies (1)

Headers (11)

Test Results (2/2)

Filter Results

PASSED

Test for status code 201

PASSED

Test for status message : Created

QA_Telecom

Fork 0

Share

Variable	Value
BaseUrl	https://thinking-tester-contact-list.herokuapp.com
userEmail	test1231774581072217@gmail.com
Token1	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJfaWQiOiI2OWM1ZjU1MjFIMmlwODAwM...
Add variable	

Test Case 2 : Get User Profile

ContactListApplication / GetUserProfile

SaveShare

GET{{BaseUrl}}/users/meSend

DocsParamsAuthorizationHeaders (8)BodyScriptsSettingsCool

Auth Type

Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Token

{{Token1}}

ResponseHistory

ContactListApplication / GetUserProfile

Save

GET{{BaseUrl}}/users/me

DocsParamsAuthorizationHeaders (8)BodyScriptsSettings

Pre-request

Post-response

```
1 // Test to check Status code is 200
2 pm.test("Test for status code is 200",()=>{
3   pm.expect(pm.response.code).to.eql(200);
4 })
5
6 // Test to check status message : OK
7 pm.test("Test for status message is OK",()=>{
8   pm.expect(pm.response.status).to.eql("OK")
9 })
```

Packages</> Snipp

BodyCookies (1)Headers (10)Test Results (2/2)200 OK974 ms863 B

Filter Results

PASSEDTest for status code is 200

PASSEDTest for status message is OK

Test Case 3 : Update User

HTTP ContactListApplication / **UpdateUser** Save

PATCH ▼ `{{BaseUrl}}/users/me`

Docs Params Authorization Headers (9) Body Scripts Settings

Pre-request

Post-response ●

```
1 // Validate status code should be 200
2 pm.test('Test for status code is 200', ()=>{
3   pm.expect(pm.response.code).to.eql(200);
4 })
5
6 // Validate status message should be 'OK'
7 pm.test('Test for status message is OK', ()=>{
8   pm.expect(pm.response.status).to.eql("OK");
9 })
```

Packages </> Snippet

Body Cookies Headers (10) **Test Results (2/2)** 200 OK • 1.06 s • 865 B • Save

Filter Results ▼

PASSED 'Test for status code is 200'

PASSED Test for status message is OK

Test Case 4 : Log In User

HTTP ContactListApplication / **LoginUser** Save

POST ▼ `{{BaseUrl}}/users/login`

Docs Params Authorization Headers (9) Body Scripts Settings

Pre-request

Post-response ●

```
1 // Validate status code
2 pm.test("Test for status code is 200", ()=>{
3   pm.expect()
4 })
5
6 // Validate status message : OK
7 pm.test('Test for status message is OK', () => {
8   pm.expect(pm.response.status).to.eql("OK")
9 })
10
11 // save the Token in environment variable
12 let tokenValue= pm.response.json().token;
13 pm.environment.set("Token2",tokenValue);
```

Packages </> Snippet

Body Cookies (1) Headers (11) **Test Results (2/2)** 200 OK • 1.02 s • 1.24 KB • Save

Filter Results ▼

PASSED Test for status code is 200

PASSED 'Test for status message is OK'

Test Case 5 : Add Contact

HTTP ContactListApplication / AddContact Save S

POST ▼ `{{BaseUrl}}/contacts` S

Docs Params Authorization Headers (10) Body Scripts Settings

Pre-request

Post-response ●

```
1 // Validate status code : 201
2 pm.test("Test for status code is 201", ()=>{
3   pm.expect(pm.response.code).to.eql(201);
4 })
5
6 // Validate Status message is : Created
7 pm.test("Test for tatus message is Created", ()=>{
8   pm.expect(pm.response.status).to.eql("Created");
9 })
10
11 // save Id in environment variable
12 let id = pm.response.json()._id;
13 pm.environment.set("ID", id);
```

Packages </> Snippets

Body Cookies (1) Headers (10) Test Results (2/2) 🕒 **201 Created** • 1.02 s • 1.03 KB • 🔍 💾 Save Re

Filter Results ▼

PASSED Test for status code is 201

PASSED Test for tatus message is Created

Test Case 6 : Get Contact List

HTTP ContactListApplication / GetContactList Save

GET ▼ `{{BaseUrl}}/contacts`

Docs Params Authorization Headers (8) Body Scripts Settings

Pre-request

Post-response ●

```
1 // Validate status code : 200
2 pm.test("Test for status code is 200",()=>{
3   pm.expect(pm.response.code).to.eql(200);
4 })
5
6 // Validate status message is OK
7 pm.test("Test for status message is OK",()=>{
8   pm.expect(pm.response.status).to.eql("OK");
9 })
```

Packages </> Snippets

Body Cookies (1) Headers (10) Test Results (2/2) 🔄 **200 OK** • 1.09 s • 1.33 KB • 🌐 📄 Save

Filter Results ▼

PASSED Test for status code is 200

PASSED Test for status message is OK

Test Case 7 : Get Contact

HTTP ContactListApplication / GetContact Save Sh

GET ▼ `{{BaseUrl}}/contacts/` Se

Docs Params Authorization Headers (8) Body Scripts Settings

Pre-request

Post-response ●

```
1 // Validate status code is 200
2 pm.test("Test for status code is 200",()=>{
3   pm.expect(pm.response.code).to.eql(200);
4 })
5
6 //Validate status message is OK
7 pm.test("Test for status message is OK",()=>{
8   pm.expect(pm.response.status).to.eql("OK");
9 })
```

Packages </> Snippets

Body Cookies (1) Headers (10) Test Results (2/2) 🔄 **200 OK** • 983 ms • 1.33 KB • 🌐 📄 Save Res

Filter Results ▼

PASSED Test for status code is 200

PASSED Test for status message is OK

Test Case 8 : Update Contact

ContactListApplication / UpdateContact

Save

PUT{{BaseUrl}} /contacts/ {{ID}}

DocsParamsAuthorizationHeaders (10)BodyScriptsSettings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Sche

1{

2"firstName": "Amy",

3"lastName": "Miller",

4"birthdate": "1992-02-02",

5"email": "amiller@fake.com",

6"phone": "8005554242",

7"street1": "13 School St.",

8"street2": "Apt. 5",

9"city": "Washington",

10"stateProvince": "QC",

11"postalCode": "A1A1A1",

12"country": "Canada"

BodyCookies (1)Headers (10)Test Results (3/3)

200 OK • 244 ms • 1.03 KB

Save I

Filter Results

PASSED

Test for status code is 200

PASSED

Test for status message is OK

PASSED

Test for email validation is amiller@fake.com

ContactListApplication / UpdateContact Save

PUT ▼ `{{BaseUrl}} /contacts/ {{ID}}`

Docs Params Authorization Headers (10) Body Scripts Settings

Pre-request

Post-response ●

```
1 // Validate for status code 200
2 pm.test("Test for status code is 200", ()=>{
3   pm.expect(pm.response.code).to.eql(200);
4 })
5
6 // Validate for status message : OK
7 pm.test("Test for status message is OK", ()=>{
8   pm.expect(pm.response.status).to.eql("OK")
9 })
10
11 // Validate Email "amiller@fake.com"
12 let resEmail= pm.response.json().email;
13 pm.test("Test for email validation is amiller@fake.com", ()=>{
14   pm.expect(resEmail).to.eql("amiller@fake.com");
```

200 OK • 244 ms • 1.03 KB • S

Body Cookies (1) Headers (10) Test Results (3/3)

`{}` JSON ▼ Preview Visualize ▼

```
1 {
2   "_id": "69c618d61b2b0800151807ef",
3   "firstName": "Amy",
4   "lastName": "Miller",
5   "birthdate": "1992-02-02",
6   "email": "amiller@fake.com",
7   "phone": "8005554242",
8   "status": "Active"
```

Test Case 9 : Partial Update Contact

ContactListApplication / PartialUpdateContact

Save

Sh

PATCH

{{BaseUrl}} /contacts/ {{ID}}

Se

Docs

Params

Authorization

Headers (10)

Body

Scripts

Settings

Pre-request

Post-response

```
2 pm.test("Test for status code is 200", ()=>{
3   pm.expect(pm.response.code).toEqual(200);
4 })
5
6 //Validate status message is OK
7 pm.test("Test for status message is OK", ()=>{
8   pm.expect(pm.response.status).toEqual("OK");
9 })
10
11 // Validate first name is Anna
12 let firstName= pm.response.json().firstName;
13 pm.test("Test for first name is Anna", () => {
14   pm.expect(firstName).toEqual("Anna");
15 })
```

Packages

</> Snippets

Body

Cookies (1)

Headers (10)

Test Results (3/3)

200 OK

1.03 s

1.04 KB

Save Res

Filter Results

PASSED Test for status code is 200

PASSED Test for status message is OK

PASSED Test for first name is Anna

Test Case 10 : Logout User

ContactListApplication / LogoutUser

Save

POST

{{BaseUrl}}/users/logout

Docs

Params

Authorization

Headers (9)

Body

Scripts

Settings

Pre-request

Post-response

```

1 // Validate status code is 200
2 pm.test("Test for status code is 200", ()=>{
3     pm.expect(pm.response.code).to.eql(200);
4 })
5
6 // Validate status message OK
7 pm.test("Test for status message is OK", ()=>{
8     pm.expect(pm.response.status).to.eql("OK");
9 })

```

Packages

Snippets

Body

Cookies (1)

Headers (8)

Test Results (2/2)

200 OK

1.26 s

653 B

Save

Filter Results

PASSED Test for status code is 200

PASSED Test for status message is OK

Manual Run Report

ContactListApplication - Run results

Run Again

New Run

Automate Run

Share

...

Ran today at 12:37:29 PM

View all runs

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	QA_Telecom	1	5s 437ms	22	0	334 ms

RUN SUMMARY

View Results

1

POST AddNewUser

2 | 0

GET GetUserProfile

2 | 0

PATCH UpdateUser

2 | 0

POST LoginUser

2 | 0

POST AddContact

2 | 0

GET GetContactList

2 | 0

GET GetContact

2 | 0

PUT UpdateContact

3 | 0

PATCH PartialUpdateContact

3 | 0

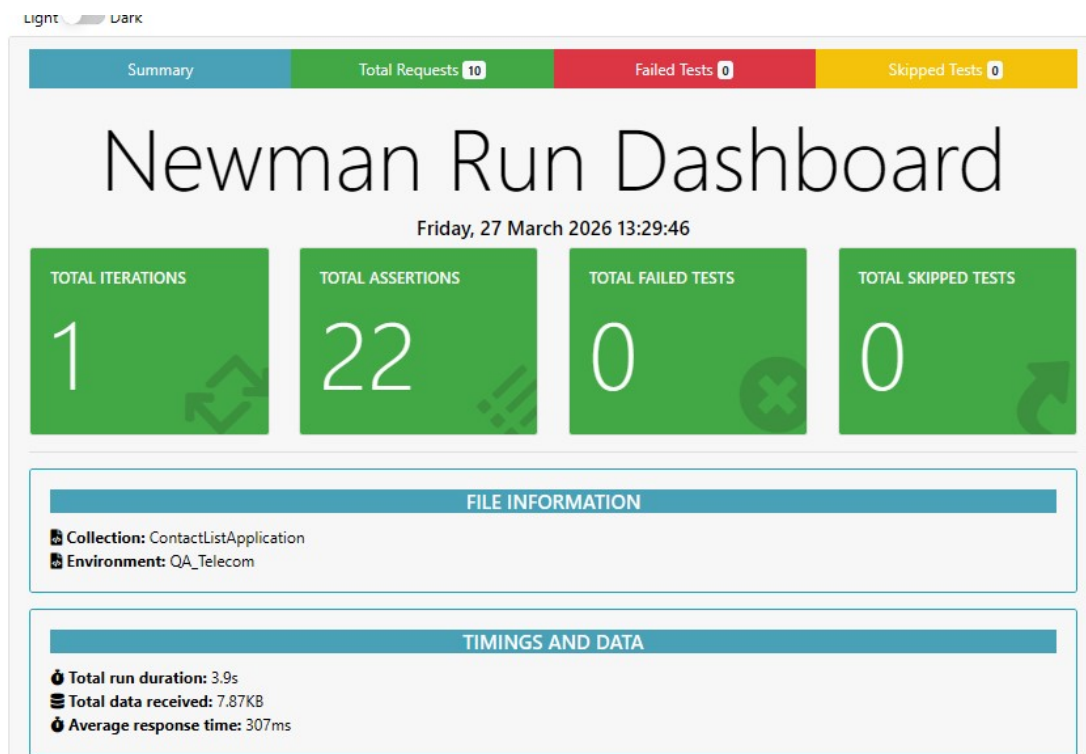
POST LogoutUser

2 | 0

Automate Run on Postman CLI

	executed	failed
iterations	1	0
requests	10	0
test-scripts	10	0
prerequest-scripts	1	0
assertions	22	0
total run duration: 4.2 s		
total data received: 4.77 KB (approx)		
average response time: 337 ms [min: 246 ms, max: 1096 ms, s.d.: 252 ms]		

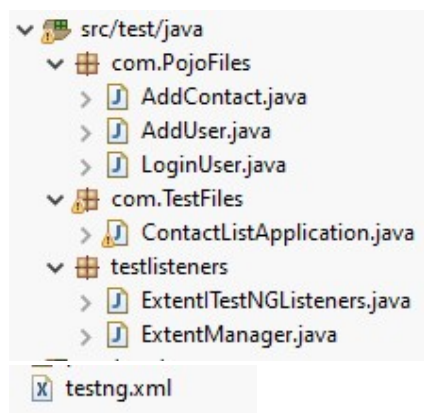
Reports with Newman



SUMMARY ITEM	TOTAL	FAILED
Requests	10	0
Prerequisite Scripts	1	0
Test Scripts	10	0
Assertions	22	0
Skipped Tests	0	-

9. RestAssured Automation Framework

Folder Structure



Maven Dependencies

```

<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>TelecomDomain_RestAssuredFramework</groupId>
  <artifactId>TelecomDomain_RestAssuredFramework</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <dependencies>
    <!-- https://mvnrepository.com/artifact/io.rest-assured/rest-assured -->
    <dependency>
      <groupId>io.rest-assured</groupId>
      <artifactId>rest-assured</artifactId>
      <version>5.5.5</version>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.json/json -->
    <dependency>
      <groupId>org.json</groupId>
      <artifactId>json</artifactId>
      <version>20220320</version>
    </dependency>

    <!-- https://mvnrepository.com/artifact/io.rest-assured/json-schema-validator -->
    <dependency>
      <groupId>io.rest-assured</groupId>
      <artifactId>json-schema-validator</artifactId>
      <version>5.1.1</version>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/com.google.code.gson/gson -->
    <dependency>
      <groupId>com.google.code.gson</groupId>
      <artifactId>gson</artifactId>
      <version>2.10.1</version>
    </dependency>

    <!-- https://mvnrepository.com/artifact/io.rest-assured/json-path -->
    <dependency>
      <groupId>io.rest-assured</groupId>
      <artifactId>json-path</artifactId>
      <version>5.1.1</version>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.testng/testng -->
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.7.1</version>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/com.github.scribejava/scribejava-apis -->
    <dependency>
      <groupId>com.github.scribejava</groupId>
      <artifactId>scribejava-apis</artifactId>
      <version>8.3.1</version>
      <scope>runtime</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/io.rest-assured/xml-path -->
    <dependency>
      <groupId>io.rest-assured</groupId>
      <artifactId>xml-path</artifactId>
      <version>5.3.0</version>
    </dependency>

    <dependency>
      <groupId>com.aventstack</groupId>
      <artifactId>extentreports</artifactId>
      <version>5.0.8</version>
    </dependency>
  </dependencies>
</project>

```

```
AddUser.java X
1  package com.PojoFiles;
2
3  public class AddUser {
4
5      /*
6       * {
7       * "firstName": "Test",
8       * "lastName": "User",
9       * "email": "test@fake.com",
10      * "password": "myPassword"
11      * }
12      * */
13
14      private String firstName;
15      private String lastName;
16      private String email;
17      private String password;
18
19      public String getFirstName() {
20          return firstName;
21      }
22      public void setFirstName(String firstName) {
23          this.firstName = firstName;
24      }
25      public String getLastName() {
26          return lastName;
27      }
28      public void setLastName(String lastName) {
29          this.lastName = lastName;
30      }
31      public String getEmail() {
32          return email;
33      }
34      public void setEmail(String email) {
35          this.email = email;
36      }
37      public String getPassword() {
38          return password;
39      }
40      public void setPassword(String password) {
41          this.password = password;
42      }
43  }
44
```

```
LoginUser.java X
1 package com.PojoFiles;
2
3 public class LoginUser {
4     /*
5      * {
6      * "email": "test2@fake.com",
7      * "password": "myNewPassword"
8      * }
9      */
10
11     private String email;
12     private String password;
13
14     public String getEmail() {
15         return email;
16     }
17     public void setEmail(String email) {
18         this.email = email;
19     }
20     public String getPassword() {
21         return password;
22     }
23     public void setPassword(String password) {
24         this.password = password;
25     }
26
27 }
28
```

```

ContactListApplication.java X
package com.PojoFiles;
package com.TelecomDomain_RestAssuredFramework/src/test/java/com/TestFiles/ContactListApplication.java
3 import org.testng.Assert;[]
18
19 Run ALL
20 public class ContactListApplication {
21     String token1;
22     String token2;
23     String ID;
24
25     @Test(priority=1)
26     Run / Debug
27     public void testAddNewUser() {
28         // create Payload
29         AddUser userData = new AddUser();
30
31         userData.setFirstName("Test");
32         userData.setLastName("User");
33         long time = System.currentTimeMillis();
34         userData.setEmail("test"+time+"@fake.com");
35         userData.setPassword("myPassword");
36
37         Response res = given()
38             .header("content-Type", "application/json")
39             .body(userData)
40
41             .when().post("https://thinking-tester-contact-list.herokuapp.com/users");
42
43         // validate status code
44         Assert.assertEquals(res.getStatusCode(), 201, "Status code not matched!!!");
45         System.out.println("Status code matched....."+res.getStatusCode());
46
47         // validate Status Message
48         Assert.assertTrue(res.getStatusLine().contains("Created"), "Status message not matched!!!");
49         System.out.println("Status message is : "+res.getStatusLine());
50
51         // store generated token
52         token1 = res.jsonPath().getString("token");
53         System.out.println("Generated token is : "+ token1);
54
55         res.then().log().body();
56         System.out.println("-----");
57     }

```

```

    }

    @Test (dependsOnMethods="testAddNewUser",priority =2)
    Run | Debug
    public void testGetUserProfile() {

        Response res = given()
            .header("Authorization","Bearer "+ token1)

            .when().get("https://thinking-tester-contact-list.herokuapp.com/users/me");

            //validate status code
            Assert.assertEquals(res.getStatusCode(),200 , "Status code not matched!!!");
            System.out.println("Status code matched...."+res.getStatusCode());

            // validate status message
            Assert.assertTrue(res.getStatusLine().contains("OK"), "Status message not matched!!!");
            System.out.println("Status message matched...."+res.getStatusLine());

            res.then().log().body();
            System.out.println("-----");

    }

    @Test(dependsOnMethods="testAddNewUser",priority=3)
    Run | Debug
    public void testUpdateUser() {

        // create payload
        AddUser userData = new AddUser();
        userData.setFirstName("Updated");
        userData.setLastName("Username");
        long time = System.currentTimeMillis();
        userData.setEmail("test2"+time+"@fake.com");
        userData.setPassword("myNewPassword");
        Response res = given()
            .header("Authorization","Bearer "+ token1)

            .when().patch("https://thinking-tester-contact-list.herokuapp.com/users/me");

            res.then().log().body();

            // validate status code
            Assert.assertEquals(res.getStatusCode(),200 , "Status code not matched!!!");
            System.out.println("Status code matched...."+res.getStatusCode());

            // validate status message
            Assert.assertTrue(res.getStatusLine().contains("OK"), "Status message not matched!!!");
            System.out.println("Status message matched...."+res.getStatusLine());
            System.out.println("-----");

    }

    @Test(priority=4)
    Run | Debug
    public void testLoginUser() {

        // create payload
        LoginUser loginData = new LoginUser();
        loginData.setEmail("test2732026@gmail.com");
        loginData.setPassword("test123");

        Response res = given()
            .header("content-Type","application/json")
            .body(loginData)

            .when().post("https://thinking-tester-contact-list.herokuapp.com/users/login");

            res.then().log().body();

            // validate status code
            Assert.assertEquals(res.getStatusCode(), 200 , "Status code not matched!!!");
            System.out.println("Status code matched...."+res.getStatusCode());

            // validate status message
            Assert.assertTrue(res.getStatusLine().contains("OK"), "Status message not matched!!!");
            System.out.println("Status message matched...."+res.getStatusLine());

            // Store generated Token
            token2 = res.jsonPath().getString("token");
            System.out.println("Generated token value is : "+token2);

            System.out.println("-----");
    }

```

```

@Test(dependsOnMethods="testLoginUser", priority=5)
Run | Debug
public void testAddContact() {

    Response res = given()
        .header("Authorization", "Bearer "+ token2)
        .contentType(ContentType.JSON)
        .body("{\n"
            + "    \"firstName\": \"John\", \n"
            + "    \"lastName\": \"Doe\", \n"
            + "    \"birthdate\": \"1970-01-01\", \n"
            + "    \"email\": \"jdoe@fake.com\", \n"
            + "    \"phone\": \"8005555555\", \n"
            + "    \"street1\": \"1 Main St.\", \n"
            + "    \"street2\": \"Apartment A\", \n"
            + "    \"city\": \"Anytown\", \n"
            + "    \"stateProvince\": \"KS\", \n"
            + "    \"postalCode\": \"12345\", \n"
            + "    \"country\": \"USA\" \n"
            + "}")
        // .body(addData)

        .when().post("https://thinking-tester-contact-list.herokuapp.com/contacts");

    res.then().log().body();

    // validate status code
    Assert.assertEquals(res.getStatusCode(), 201, "Status code not matched!!!");
    System.out.println("Status code matched....."+res.getStatusCode());

    // validate status message
    Assert.assertTrue(res.getStatusLine().contains("Created"), "Status message not matched!!!");
    System.out.println("Status message matched....."+res.getStatusLine());

    // store generated ID
    ID = res.jsonPath().getString("_id");
    System.out.println("Generated ID is : "+ID);
    System.out.println("-----");
}

@Test(dependsOnMethods="testLoginUser", priority=6)
Run | Debug
public void testGetContactList() {

    Response res = given()
        .header("Authorization", "Bearer "+token2)

        .when().get("https://thinking-tester-contact-list.herokuapp.com/contacts");

    res.then().log().body();

    // validate status code
    Assert.assertEquals(res.getStatusCode(), 200, "Status code not matched!!!");
    System.out.println("Status code matched....."+res.getStatusCode());
    // validate status message
    Assert.assertTrue(res.getStatusLine().contains("OK"), "Status message not matched!!!");
    System.out.println("Status message matched....."+res.getStatusLine());
}

@Test(dependsOnMethods="testAddContact", priority=7)
Run | Debug
public void testGetContact() {

    Response res = given()
        .header("Authorization", "Bearer "+token2)

        .when().get("https://thinking-tester-contact-list.herokuapp.com/contacts/"+ID);

    res.then().log().body();

    // validate status code
    Assert.assertEquals(res.getStatusCode(), 200, "Status code not matched!!!");
    System.out.println("Status code matched....."+res.getStatusCode());
    // validate status message
    Assert.assertTrue(res.getStatusLine().contains("OK"), "Status message not matched!!!");
    System.out.println("Status message matched....."+res.getStatusLine());
}

```

```
@Test(dependsOnMethods="testAddContact", priority = 8)
```

```
Run | Debug
```

```
public void testUpdateContact() {
```

```
    Response res = given()
```

```
        .header("Authorization", "Bearer "+token2)
```

```
        .contentType(ContentType.JSON)
```

```
        .body("{\n"
```

```
            + "    \"firstName\": \"Amy\", \n"
```

```
            + "    \"lastName\": \"Miller\", \n"
```

```
            + "    \"birthdate\": \"1992-02-02\", \n"
```

```
            + "    \"email\": \"amiller@fake.com\", \n"
```

```
            + "    \"phone\": \"8005554242\", \n"
```

```
            + "    \"street1\": \"13 School St.\", \n"
```

```
            + "    \"street2\": \"Apt. 5\", \n"
```

```
            + "    \"city\": \"Washington\", \n"
```

```
            + "    \"stateProvince\": \"QC\", \n"
```

```
            + "    \"postalCode\": \"A1A1A1\", \n"
```

```
            + "    \"country\": \"Canada\" \n"
```

```
            + "}")
```

```
    .when().put("https://thinking-tester-contact-list.herokuapp.com/contacts/"+ID);
```

```
    res.then().log().body();
```

```
    // validate status code
```

```
    Assert.assertEquals(res.getStatusCode(), 200, "Status code not matched!!!");
```

```
    System.out.println("Status code matched....."+res.getStatusCode());
```

```
    // validate status message
```

```
    Assert.assertTrue(res.getStatusLine().contains("OK"), "Status message not matched!!!");
```

```
    System.out.println("Status message matched....."+res.getStatusLine());
```

```
    // validate Email
```

```
    String email = res.jsonPath().getString("email");
```

```
    Assert.assertEquals(email, "amiller@fake.com", "Update Failed....Email not matched!!! ");
```

```
    System.out.println("Update successful....Email is : "+email);
```

```
}
```

```
@Test(dependsOnMethods="testUpdateContact", priority=9)
```

```
Run | Debug
```

```
public void testPartialUpdateContact() {
```

```
    Response res = given()
```

```
        .header("Authorization", "Bearer "+token2)
```

```
        .contentType(ContentType.JSON)
```

```
        .body("{\n"
```

```
            + "    \"firstName\": \"Anna\" \n"
```

```
            + "}")
```

```
    .when().patch("https://thinking-tester-contact-list.herokuapp.com/contacts/"+ID);
```

```
    res.then().log().body();
```

```
    // validate status code
```

```
    Assert.assertEquals(res.getStatusCode(), 200, "Status code not matched!!!");
```

```
    System.out.println("Status code matched....."+res.getStatusCode());
```

```
    // validate status message
```

```
    Assert.assertTrue(res.getStatusLine().contains("OK"), "Status message not matched!!!");
```

```
    System.out.println("Status message matched....."+res.getStatusLine());
```

```
    // validate first name : Anna
```

```
    String fName= res.jsonPath().getString("firstName");
```

```
    Assert.assertEquals(fName, "Anna", "Update Failed....First Name not matched!!!");
```

```
    System.out.println("Partial Update successful....First Name is : "+fName);
```

```
}
```

```
@Test(priority=10)
```

```
Run / Debug
```

```
public void testLogoutUser() {
```

```
    Response res = given()
```

```
        .header("Authorization", "Bearer "+token2)
```

```
        .when().post("https://thinking-tester-contact-list.herokuapp.com/users/logout");
```

```
    res.then().log().body();
```

```
    // Validate status code
```

```
    Assert.assertEquals(res.getStatusCode(),200,"Status code not matched!!!");
```

```
    System.out.println("Status code is : "+res.getStatusCode());
```

```
    // Validate status message
```

```
    Assert.assertTrue(res.getStatusLine().contains("OK"), "Status message not matched!!!");
```

```
    System.out.println("Status message matched....."+res.getStatusLine());
```

```
}
```

```
*ExtentTestNGListeners.java X
```

```
1 package testlisteners;
```

```
2
```

```
3 import org.testng.ITestContext;
```

```
10 public class ExtentTestNGListeners implements ITestListener {
```

```
11 ExtentReports extent=ExtentManager.getInstance();
```

```
12
```

```
13 ThreadLocal<ExtentTest> parentTest=new ThreadLocal<ExtentTest>();
```

```
14
```

```
15 public void onTestStart(ITestResult result)
```

```
16 {
```

```
17     ExtentTest extentTest=extent.createTest(result.getMethod().getMethodName());
```

```
18     parentTest.set(extentTest);
```

```
19 }
```

```
20 public void onTestSuccess(ITestResult result) {
```

```
21
```

```
22     //parentTest.get().pass("Test Passed");
```

```
23     parentTest.get().log(Status.PASS,result.getName()+"Got Successfully Pass!");
```

```
24 }
```

```
25 public void onTestFailure(ITestResult result)
```

```
26 {
```

```
27     parentTest.get().fail("Test Failed "+result.getThrowable().getMessage());
```

```
28     System.out.println("Test Failure : "+result.getMethod().getMethodName());
```

```
29     parentTest.get().log(Status.FAIL,result.getName()+"Test Fail");
```

```
30 }
```

```
31 public void onTestSkipped(ITestResult result) {
```

```
32     parentTest.get().skip("Test Skipped "+result.getThrowable().getMessage());
```

```
33 }
```

```
34 public void onFinish(ITestContext context) {
```

```
35     extent.flush();
```

```
36 }
```

```
37 }
```

```
38
```

```
39
```

```
40
```

```
41
```

```
42
```

```
43
```

```
44
```

```
45
```

```
46
```

```
47
```

```
48
```

```
49
```

```
50
```

```
51
```

```
52
```

```

ExtentManager.java X
1 package testlisteners;
2
3 import java.text.SimpleDateFormat;
4
5 public class ExtentManager {
6
7     public static ExtentReports extent;
8
9     public static ExtentReports getInstance()
10    {
11        if(extent==null)
12        {
13            extent=createInstance();
14            return extent;
15        }
16        else
17        {
18            return extent;
19        }
20    }
21
22    static String rname;
23    public static ExtentReports createInstance()
24    {
25        String timestamp=new SimpleDateFormat("yyyy.MM.dd.HH.mm.ss").format(new Date());
26        rname="TelecomDomainTestReport"+timestamp+".html";
27
28        ExtentSparkReporter sparkReporter=new ExtentSparkReporter(System.getProperty("user.dir")+"/TelecomDomain_TestReports/"+rname);
29        sparkReporter.config().setTheme(Theme.DARK);
30        sparkReporter.config().setReportName("TelecomDomainTestReport");
31        sparkReporter.config().setDocumentTitle("TelecomDomain_Report");
32
33        ExtentReports extent=new ExtentReports();
34        extent.attachReporter(sparkReporter);
35
36        return extent;
37    }
38 }
39
40
41
42
43
44
45
46
47
48
49
50

```

```

testng.xml X
1 https://testng.org/testng-1.0.dtd (doctype)
2 <?xml version="1.0" encoding="UTF-8"?>
3 <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
4 <suite name="Suite">
5     <listeners>
6         <listener class-name="testlisteners.ExtentITestNGListeners"/>
7     </listeners>
8     <test thread-count="5" name="Test">
9         <classes>
10            <class name="com.TestFiles.ContactListApplication"/>
11        </classes>
12    </test> <!-- Test -->
13 </suite> <!-- Suite -->
14
15
16
17

```

Console Screenshot :

```
[RemoteTesting] detected TestNG version 7.7.1
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Status code matched.....201
Status message is : HTTP/1.1 201 Created
Generated token is : eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJfYWQioiI2OWM3ZGRmNjgxMzZlYzAwMTU5MDRlMDIiLCJpYXQiojE3NzQ3MDYxNjZ9.TK6S
{
  "user": {
    "id": "69c7ddf68136ec0015904b02",
    "firstName": "Test",
    "lastName": "User",
    "email": "test1774706138179@fake.com",
    "__v": 1
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJfYWQioiI2OWM3ZGRmNjgxMzZlYzAwMTU5MDRlMDIiLCJpYXQiojE3NzQ3MDYxNjZ9.TK6S"
}
-----
Status code matched....200
Status message matched....HTTP/1.1 200 OK
{
  "id": "69c7ddf68136ec0015904b02",
  "firstName": "Test",
  "lastName": "User",
  "email": "test1774706138179@fake.com",
  "__v": 1
}
-----
{
  "id": "69c7ddf68136ec0015904b02",
  "firstName": "Test",
  "lastName": "User",
  "email": "test1774706138179@fake.com",
  "__v": 1
}
Status code matched.....200
Status message matched.....HTTP/1.1 200 OK
-----
{
  "user": {
    "id": "69c5ed611b2b0800151807b3",
    "firstName": "Sumana",
    "lastName": "Chowdhury",
    "email": "test2732026@gmail.com",
    "__v": 48
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJfYWQioiI2OWM1ZWQ2MTFmImIwODAwMTUxODAzYjMlLCJpYXQiojE3NzQ3MDYxNjZ9.i"
}
Status code matched.....200
Status message matched.....HTTP/1.1 200 OK
Generated token value is : eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJfYWQioiI2OWM1ZWQ2MTFmImIwODAwMTUxODAzYjMlLCJpYXQiojE3NzQ3MDYxNjZ9.i
-----
{
  "id": "69c7de018136ec0015904b04",
  "firstName": "John",
  "lastName": "Doe",
  "birthdate": "1970-01-01",
  "email": "jdoe@fake.com",
  "phone": "8005555555",
  "street1": "1 Main St.",
  "street2": "Apartment A",
  "city": "Anytown",
  "stateProvince": "KS",
  "postalCode": "12345",
  "country": "USA",
  "owner": "69c5ed611b2b0800151807b3",
  "__v": 0
}
Status code matched.....201
Status message matched.....HTTP/1.1 201 Created
Generated ID is : 69c7de018136ec0015904b04
-----
r
```

```

[
  {
    "_id": "69c6186b93bb9a001552219a",
    "firstName": "John",
    "lastName": "Doe",
    "birthdate": "1970-01-01",
    "email": "jdoe@fake.com",
    "phone": "8005555555",
    "street1": "1 Main St.",
    "street2": "Apartment A",
    "city": "Anytown",
    "stateProvince": "KS",
    "postalCode": "12345",
    "country": "USA",
    "owner": "69c5ed611b2b0800151807b3",
    "__v": 0
  },
  {
    "_id": "69c7ba3575b6e80015acac5c",
    "firstName": "John",
    "lastName": "Doe",
    "birthdate": "1970-01-01",
    "email": "jdoe@fake.com",
    "phone": "8005555555",
    "street1": "1 Main St.",
    "street2": "Apartment A",
    "city": "Anytown",
    "stateProvince": "KS",
    "postalCode": "12345",
    "country": "USA",
    "owner": "69c5ed611b2b0800151807b3",
    "__v": 0
  },
],
[
  {
    "_id": "69c638f193bb9a0015522226",
    "firstName": "Anna",
    "lastName": "Miller",
    "birthdate": "1992-02-02",
    "email": "amiller@fake.com",
    "phone": "8005554242",
    "street1": "13 School St.",
    "street2": "Apt. 5",
    "city": "Washington",
    "stateProvince": "QC",
    "postalCode": "A1A1A1",
    "country": "Canada",
    "owner": "69c5ed611b2b0800151807b3",
    "__v": 0
  },
  {
    "_id": "69c7dbd175b6e80015acac7c",
    "firstName": "Anna",
    "lastName": "Miller",
    "birthdate": "1992-02-02",
    "email": "amiller@fake.com",
    "phone": "8005554242",
    "street1": "13 School St.",
    "street2": "Apt. 5",
    "city": "Washington",
    "stateProvince": "QC",
    "postalCode": "A1A1A1",
    "country": "Canada",
    "owner": "69c5ed611b2b0800151807b3",
    "__v": 0
  }
]

```

```

}
Status code matched.....200
Status message matched.....HTTP/1.1 200 OK
{
  "_id": "69c7de018136ec0015904b04",
  "firstName": "John",
  "lastName": "Doe",
  "birthdate": "1970-01-01",
  "email": "jdoe@fake.com",
  "phone": "8005555555",
  "street1": "1 Main St.",
  "street2": "Apartment A",
  "city": "Anytown",
  "stateProvince": "KS",
  "postalCode": "12345",
  "country": "USA",
  "owner": "69c5ed611b2b0800151807b3",
  "__v": 0
}
Status code matched.....200
Status message matched.....HTTP/1.1 200 OK
{
  "_id": "69c7de018136ec0015904b04",
  "firstName": "Amy",
  "lastName": "Miller",
  "birthdate": "1992-02-02",
  "email": "amiller@fake.com",
  "phone": "8005554242",
  "street1": "13 School St.",
  "street2": "Apt. 5",
  "city": "Washington",
  "stateProvince": "QC",
  "postalCode": "A1A1A1",
  "country": "Canada",
  "owner": "69c5ed611b2b0800151807b3",
  "__v": 0
}
Status code matched.....200
Status message matched.....HTTP/1.1 200 OK
Update successful....Email is : amiller@fake.com
{
  "_id": "69c7de018136ec0015904b04",
  "firstName": "Anna",
  "lastName": "Miller",
  "birthdate": "1992-02-02",
  "email": "amiller@fake.com",
  "phone": "8005554242",
  "street1": "13 School St.",
  "street2": "Apt. 5",
  "city": "Washington",
  "stateProvince": "QC",
  "postalCode": "A1A1A1",
  "country": "Canada",
  "owner": "69c5ed611b2b0800151807b3",
  "__v": 0
}
Status code matched.....200
Status message matched.....HTTP/1.1 200 OK
Partial Update successful....First Name is : Anna

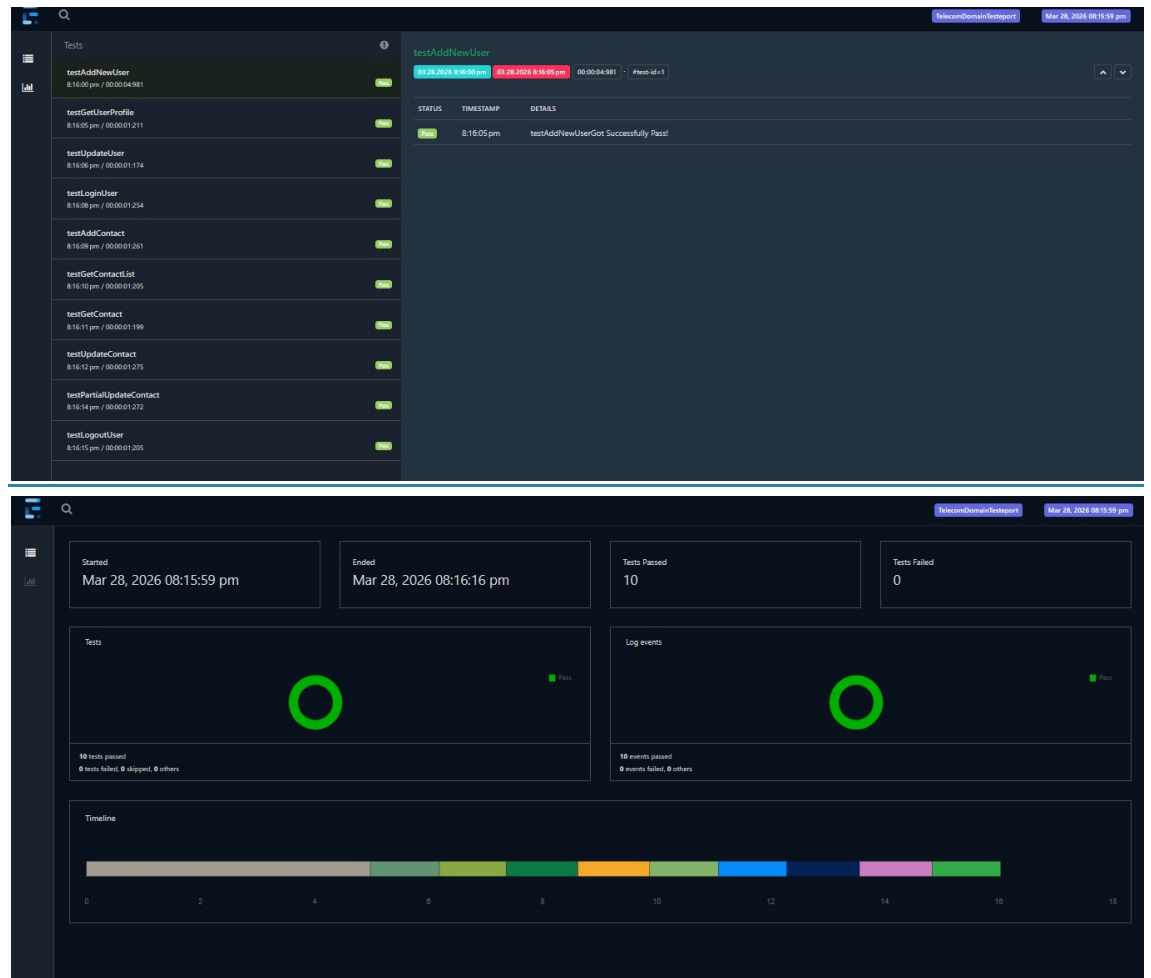
Status code is : 200
'Status message matched.....HTTP/1.1 200 OK
PASSED: com.TestFiles.ContactListApplication.testGetContact
PASSED: com.TestFiles.ContactListApplication.testLoginUser
PASSED: com.TestFiles.ContactListApplication.testAddContact
PASSED: com.TestFiles.ContactListApplication.testLogoutUser
PASSED: com.TestFiles.ContactListApplication.testGetContactList
PASSED: com.TestFiles.ContactListApplication.testPartialUpdateContact
PASSED: com.TestFiles.ContactListApplication.testUpdateUser
PASSED: com.TestFiles.ContactListApplication.testUpdateContact
PASSED: com.TestFiles.ContactListApplication.testAddNewUser
PASSED: com.TestFiles.ContactListApplication.testGetUserProfile

=====
Default test
Tests run: 10, Failures: 0, Skips: 0
=====

=====
Default suite
Total tests run: 10, Passes: 10, Failures: 0, Skips: 0
-----

```

10. Reporting



11. Conclusion

The API testing of the Telecom Contact List Application was successfully completed using **Postman with Chai assertions** and the **RestAssured framework**. The testing validated the functionality of all major APIs, including authentication and contact management operations.